Adroit Technologies Protocol Driver

Name	Modbus RTU Ethernet TCP/IP
DLL	MOD_ETH

COPYRIGHT

Advanced Worx 112 (Pty) Ltd
20 Waterford Office Park
189 Witkoppen Road (c/o Waterford Drive)
Fourways
South Africa

https://adroittech.co.za/

Contents

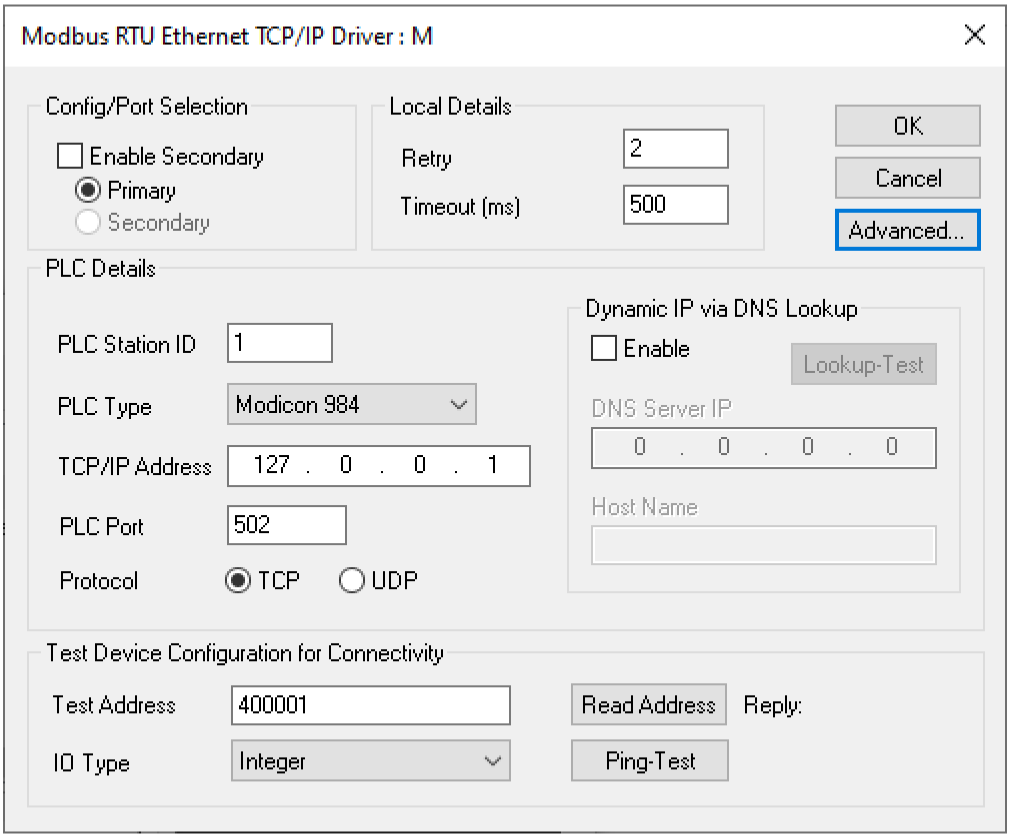
1. Overview

Adroit supports communication with the following PLCs: Modicon 84, 884, 184, 384, 484, 584, 984, Quantum, Premium, M340, Ethernet/serial gateways and many other third-party devices over TCP/IP.

2. Device Configuration

2.1. Device Configuration

PLEASE NOTE
Changes are only accepted upon pressing the final OK button on the main device configuration dialog below. Agent server may need to be restarted for changes to take effect.



Advanced Button – Access advanced communication parameters and driver settings for this device.

2.1.1. Config/Port Selection

Enable Secondary – (Default = unchecked) Enables secondary configuration for dual redundant PLCs, via **Primary** and **Secondary** radio buttons. (Do not enable if a secondary channel is not configured correctly.)

2.1.2. PLC Details

PLC Station ID - Valid Station number. See section Protocol Definition and Considerations - 6.1 Address Padding for definition of address padding for addresses greater than 254.

PLC Type - Applicable PLC types

PLC IP/Address – The TCP/IP address of the PLC

PLC Port – (Default = port 502) The port address of the PLC.

Protocol – (Default = TCP) The Polling port is usually TCP , but the driver can support UDP, this is for the MASTER polling port only, it does not apply to the listener port in the advanced settings.

	PLEASE NOTE
For multiple logical mappings to one PLC (as multiple devices), use the same IP and Port number. The driver will only open multiple sockets (parallel connections) to this physical device if the "Create multiple connections..." checkbox is selected in the Advanced Settings dialog. Protocol (UDP) is not available on the advanced listener port for incoming async sockets, that is strictly TCP.	

2.2. Dynamic IP via DNS Lookup

When Dynamic IP via DNS is enabled, the driver will launch a thread that will attempt to lookup an IP address when retries are exhausted and found to be failing. This lookup process will continue indefinitely until the Agent server is stopped, it will not block a connection or the Agent server read/writes which will continue existing connection parameters until changed by a successful lookup, which will then save those new parameters.

This mode can be used on connections that are on the internet and have a dynamic IP such as when using ADSL that change IP address once a day. The device will need to be registered on a DNS server such as DYNDNS.ORG

Enable – (Default = unchecked) This will enable the DNS Lookup mechanism.

DNS Server IP – IP address of the DNS server that will handle lookup requests.

Host Name – The name of the machine/PLC registered on the DNS server. This name is used to request the associated IP address. The IP address returned will be stored in the IP Address field dynamically in the PLC Details.

2.3. Local Details

Retry - The number of attempts that the Master will attempt to communicate with the PLC before failing a transaction.

Timeout - The time that the Master will wait for a valid response from a PLC before failing an individual transaction (Read or Write).

2.4. Test Device Config for Connectivity

This section is available to help users determine whether the settings they have entered are valid and Adroit can communicate with your device. The Agent server is not required to be running.

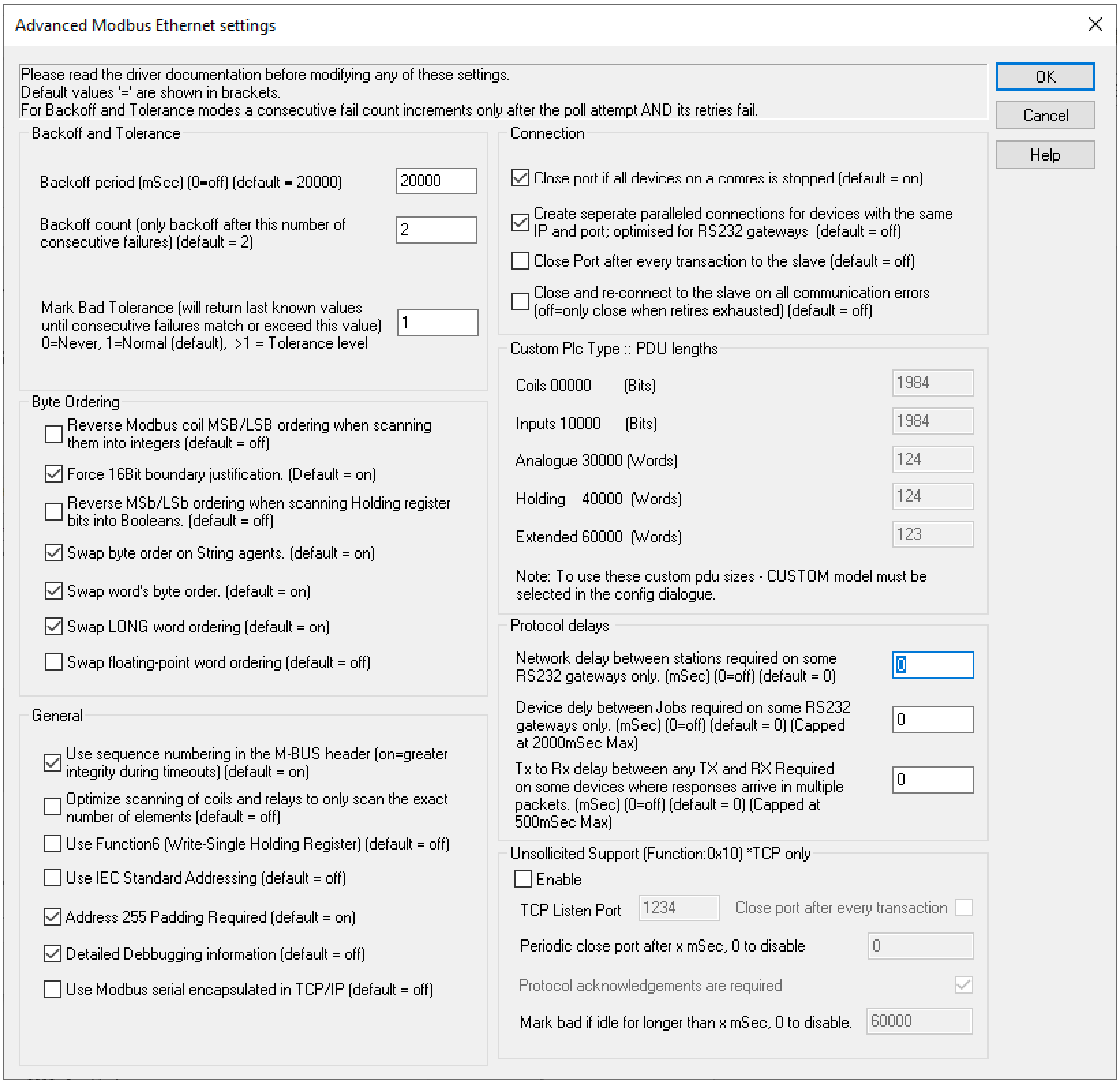
Test Address – An address in your device which will be polled when the "Read Address" button is clicked. The address entered must be valid and will match the IEC or Standard Modbus addressing scheme selected in the advanced dialogue.

IO Type – Defines the base address type to be read, i.e., Boolean, Integer, Real or String80.

Read Address – Click this button to poll the PLC with the current settings in the dialog.

Reply – The successful or failed result will be updated to this static text control.

Ping Test – Click to verify that the TCP/IP address entered into the boxes can be accessed using a ping. An appropriate MessageBox will appear as to the result.



2.5. Advanced Settings Dialog

PLEASE NOTE

The registry is only updated upon pressing the final OK button on the main device configuration dialog (the current dialog).

INFORMATION

For all configuration settings: They are offline changes to the registry and are only reloaded at start-up (hence why we suggest a restart of the Agent server). There is a case where after a read completely timeouts and exceeding all the retries as well, the driver will dynamically reload the registry and a restart is then not required. This may not be entirely apparent so when in doubt it is suggested to restart the Agent server regardless just to make sure.

2.5.1. Backoff and Tolerance Settings

Backoff period (mSec)(0=off) – The period in milliseconds that the driver will backoff from any further polling to a specific device, without affecting communications to any other device that may be on the same link. The driver will prevent any further polls to an affected device during the backoff period, after which it will attempt to re-establish communications with a single transaction attempt. If this fails, it will wait again for this period before attempting another transaction check and so on. Set backoff period to 0 to disable this function.

Backoff count (only backoff after this number of consecutive complete failures including retries) – Backoff count is the count of consecutive fail counts that are required before backoff mode start for this device. A consecutive fails count is defined as a poll request and all its retries also failing, which will increment it by 1. For example, if set to 2 and the device has 3 retries, the system will only start to backoff after the 9th consecutive failure [2 backoff count × (3 retries + 1 transactions)]. If at any time a transaction succeeds, then the backoff count is reset to 0 and the process starts over from 0.

Mark Bad Tolerance (will return last known values until consecutive failures match or exceed this value) (0 –NEVER, 1 NORMAL OPERATION, >1 TOLERANCE LEVEL) – Use this mode to reduce the frequency of bad marking of a device's associated tags during small communication interruptions. If there is a tolerant period specified greater than 1, the driver will return the last known good states and not immediately flag a failure until the tolerant count is exceeded. A consecutive fails count is defined as a poll request and all its retries also failing which will increment it by 1. Once the count exceeds the tolerance count, the fail will be reported and tags marked bad. The default state of 1 will result in fails being reported as they occur, 0 means the driver will never have its tags marked bad even if failed for extended periods.
(Please use with caution and do not set the tolerant period too high. Between two and four is recommended.)

2.5.2. Byte Ordering

Reverse Modbus coil MSB/LSB ordering when scanning them into integers – If Integer or real Agents are scanned to coil(0xxxxx) or input(1xxxxx) region and the result appears jumbled, use this setting to reverse the ordering and swap the endian order.

Force 16bit boundary justification. – (Default = checked) If checked, the driver will force Integer or real Agents scanned to coil(0xxxxx) or input(1xxxxx) region to be mapped into 16-bit word boundaries.

Reverse MSByte/LSByte ordering when scanning Holding register bits into Booleans. - Before unpacking to Boolean Agents, the word data returned from a holding register is first byte swapped.

Swap byte ordering on strings agents – Toggle to change the byte order of each word that the string occupies, for example "EHLL1O" --> "HELLO1".

Swap word's byte order – Toggle to change the byte order of word 16-bit register results.

Swap LONG word ordering – Toggle to change the word order of long 32-bit register results.

Swap floating-point value ordering – Toggle to change the word order of floating 32-bit register results. Must be selected for communication to all genuine Modicon equipment.

2.5.3. General

Use sequence numbering in the M-BUS header – (Default = checked) Increases data integrity if using a slow and noisy media or getting timeouts. This option may be incompatible with simple Modbus slaves which do not copy the frame reference bytes in the open-MBUS header.

Optimise scanning of Coils and relays to only scan the exact number of elements – Turn on this option to prevent the driver from reading multiples of 8 bits (at least 1 byte) and instead read only the required number of bits.

Use function6 (Write Single Holding Register) – (Default = unchecked) This will use function 06 to write single registers instead of writing one register using multiple register write function. Some PLCs only support function 06.

Detailed debugging information – (Default = unchecked) This causes the driver to add more debugging information to the datascope, such as data values and more specific information relating to each protocol message.

Enable Unsolicited support - (Default = checked) If checked, it will enable the driver to 'listen' for incoming (holding register writes) unsolicited data on local Port 502. However, the driver will ignore non-scanned data points.

Use IEC Standard Addressing - (Default = checked) If checked, it will enable the driver to use IEC addressing when validating, for example %MW0 instead of 400001. %MW0 = 400001.

Address 255 Padding required - (Default = checked) If checked, it will force the driver to use a 16-bit address word prefixed with a 0xFF pad byte when the address is larger than 254 (decimal).

See **Address padding** below for examples.

Use Modbus serial encapsulation in TCP/IP - (Default = unchecked) If checked, it will remove the TCP header and add a CRC to match the serial Modbus protocol. This data is encapsulated into the data portion of the IP packet and sent to a dumb TCP to serial convertor/gateway where the data portion is simple retransmitted on its secondary serial bus.

> **PLEASE NOTE**
>
> Some gateways are intelligent and do not need the driver to prepare the Modbus packet. You may need to investigate the device specifications to see if it prepares this for you, or you may need to experiment with this setting enabled and disabled.

2.5.4. Custom PLC type: PDU lengths

> **PLEASE NOTE**
>
> This is an advanced feature. Changing these settings without special consideration can break the communication altogether.

Used in conjunction with the "CUSTOM" plc type, set the appropriate limits here, if the device type is anything other than custom, the custom values will be ignored. Length count is in bits for Outputs(coils) and inputs, and 16bit registers for the others.

Outputs 000000, Inputs 100000, It is best to have length in multiples of 8, i.e. 16 will result in 2 data bytes returned in the data portion of the PDU. Some bit-devices don't respond correctly if not using a multiple (a minimum of 1 and maximum of 1984).

Analogue 300000, Holding 400000, Extended holding 600000 16-bit register types; any number can be used (a minimum of 2 and maximum of 124 registers).
Please note: Analogues and long datatypes require 2 Modbus 16bit registers per item so if these are used make sure the PDU's can contain an even number of items.

2.5.5. Connection

Close Port if all devices on a comres is stopped (default = on) – The socket will be closed if all devices sharing the communications resource object are stopped.

Create separate paralleled connections for devices with same IP and port, Optimised for Ethernet to RS232 gateways – (Default = unchecked) This setting will force the driver to use the same socket connection and thus the same communications resource in the driver for multiple devices to connect to the same IP. Turn this setting on for all devices on the same IP and port address that should share the connection. Sharing one connection forces the driver to serialize communications to a gateway, which slows the communications, but is necessary if the gateway has contains all the devices on one serial port.

Close Port after every transaction to the slave – This setting should only be used to keep the connection closed after successful communication, typically used on GPRS networks that will close the connection anyway. Now with this setting, the driver will expect it and re-open the connection when it wants to communicate again.

Close and re-connect to the slave on all communication errors – This setting should only be used to force the driver to reset the connection to the host in between every failed attempt and/or retry. If this setting is unchecked, the driver will behave normally only resetting the connection once after complete failure and all associated retries are exhausted.

2.5.6. Protocol delays

Network delay between stations required on some RS232 gateways only. (mSec) (0=off) – Inter-station delay is only applied between transactions being sent to different stations, i.e., if the last transaction station ID is not the same as the current transactions ID, a delay is enforced. Please note this only applies to devices sharing a communications resource (same IP and same Port.)

Device delay between jobs required on some RS232 gateways only. (mSec) (0=off) (Capped at 2000ms max) – Inter-transaction delay is applied between any transactions being sent. Typically network latency can be switched off as this latency will also be applied between stations. Please note this only applies to devices sharing a communications resource (same IP and same Port.)

Tx to Rx delay between any TX and RX required on some devices where responses arrive in multiple packets. (mSec) (0=off) (Capped at 500ms max) – Delay is applied between any request and response check. Typically, this is not needed as the protocol specifies a response must come in in one packet, but we do find devices that break this rule, so applying this delay will allow a small window for all the data to arrive. Otherwise, a short read and purge runaway cycle can be noted in the datascope. Start at 10 msec and work your way higher if needed.

2.5.7. Unsolicited support (Function: 0x10) *TCP Only

The driver will listen for incoming connections on the TCP listen port, and decode, if the Modbus address matches on any incoming unsolicited function 0x10(Write multiple holding register) writes. This can be considered the driver supports some SLAVE mode functionality.
The Driver will then update any holding register address that it can find as if it was a response to a Function 0x03(Read multiple holding register read).

If the device is stopped, any unsolicited messages will be discarded.

Dynamic config is supported so changing a port number will be refreshed from the registry if the device is failing to connect, or stopped and restarted.

PLEASE NOTE

It is recommended to NOT poll and use UNSOLICTED MODE simultaneously due to race conditions and potential update anomalies. For read only simple value data this will function perfectly.
Scan all jobs items with ZERO scan rate!

Enable - (Default = unchecked) The device driver will create a listen thread on the listen port

Listen port - (Default = 1234) The TCP/IP port that the driver will listen for unsolicited incoming function 0x10(hex) writes.

Close port after each transaction - (Default = unchecked) If checked, the driver will close the port after any transaction, the PLC is expected to reconnect at a later time which could be milliseconds to hours or even days.

Periodic close port after x milliseconds , 0 to disable - (Default = 0) Independently of any other functionality, the driver will simply close the port once this time has expired, then it will expect to be recontacted. This period will then reset and begin counting again.

Protocol acknowledgements are required - (Default = checked) If checked, the driver will send a simple function 0x10 acknowledgement after receiving the write.

Mark bad if idle for longer than x milliseconds , 0 to disable - (Default = 60000) If no unsolicited writes occur in this period the driver will mark all jobs bad, if hybrid mode is used these will also be marked bad, this period will then reset and begin counting again.
A poll reply will also reset this period timer.

WARNING

A successful response to any solicited (MASTER mode) protocol message will reset the mark bad idle timer to start timing again.

3. Supported Addresses

3.1. Using Modbus Addressing:

Device		Min Range	Max Range	PLC Data Type	Boolean	Integer	Real	String
Outputs	1	000001	065535	Digital	000001			
	1			Decimal		000001	000001	
	2			Signed Integer		000001 I	000001 I	
	3			BCD16		000001 B	000001 B	
Inputs	4	100001	165535	Digital	100001			
	4			Decimal		100001		
	5			Signed Integer		100001 I	100001 I	
	6			BCD16		100001 B	100001 B	
Input Registers	7	300001	365535	Digital	300001.1			
	7			Decimal		300001	300001	
	8			Signed Integer		300001 I	300001 I	
	9			BCD16		300001 B	300001 B	
Holding Registers	10	400001	465535	Digital	400001.1			
	10			Decimal		400001	400001	
	11			Signed Integer		400001 I	400001 I	
	12			BCD16		400001 B	400001 B	
	13			Long Decimal		400001 L	400001 L	
	14			Float			400001 F	
	15			Pascal-String				400001 S20
	16			Char String				400001 C20
Extended Registers	17	600000	698303	Digital	600000.1			
	17			Decimal		600000	600000	
	18			Signed Integer		600000 I	600000 I	
	19			BCD16		600000 B	600000 B	
	20			Long Decimal		600000 L	600000 L	
	21			Float			600000 F	
	22			Pascal-String				600001 S20
	23			Char String				600000 C20

3.2. Using IEC Addressing:

Device		Min Range	Max Range	PLC Data Type	Boolean	Integer	Real	String
Outputs	1	0	65535	Digital	%Q0			
	1			Decimal		%Q0	%Q0	
	2			Signed Integer		%Q0 I	%Q0 I	
	3			BCD16		%Q0 B	%Q0 B	
Inputs	4	0	65535	Digital	%I0			
	4			Decimal		%I0		
	5			Signed Integer		%I0 I	%I0 I	
	6			BCD16		%I0 B	%I0 B	
Input Registers	7	0	65535	Digital	%IW0.1			
	7			Decimal		%IW0	%IW0	
	8			Signed Integer		%IW0 I	%IW0 I	
	9			BCD16		%IW0 B	%IW0 B	
Holding Registers	10	0	65535	Digital	%MW0.1			
	10			Decimal		%MW0	%MW0	
	11			Signed Integer		%MW0 I	%MW0 I	
	12			BCD16		%MW0 B	%MW0 B	
	13			Long Decimal		%MW0 L	%MW0 L	
	14			Float			%MW0 F	
	15			Pascal-String				%MW0 S20
	16			Char String				%MW0 C20
Extended Registers	17	0	98303	Digital	%ME0.1			
	17			Decimal		%ME0	%ME0	
	18			Signed Integer		%ME0 I	%ME0 I	
	19			BCD16		%ME0 B	%ME0 B	
	20			Long Decimal		%ME0 L	%ME0 L	
	21			Float			%ME0 F	
	22			Pascal-String				%ME0 S20
	23			Char String				%ME0 C20

Note 1: Writes are not supported to inputs 100001 through to 165535, and Input registers 300001 through 365535 as these are read only mappings.

Note 2: Unsolicited support for Holding registers (400001) region only. When the PLC sends unsolicited data only those holding registers that are mapped into Adroit will be updated if they are not mapped Adroit will ignore those values.

Note 3: Char strings do not use a prefix length register so each register holds 2 characters and can be considered to be a simple array of word registers each containing 2 characters. Pascal strings are prefixed by 1 register to hold the actual length of the string. This is NOT the same as the scanning address length which indicates the maximum length.

WARNING

If 000001 (Output) region is mapped as an Integer or Real for say a Marshal Agent, you will be using function "0x15" – force multiple coils, which will write 16 coils at once, possibly overwriting some unintentionally. That means you can inadvertently control the plant. Make sure you are aware of this before mapping coils to Integers.

See Advanced Registry Settings below for details on swapping byte order for this type of mapping to coils and inputs.

3.3. Third Party Modbus Emulators

Several hardware manufacturers have adopted the Modbus protocol to communicate to their hardware. These devices use various hardware interfaces that have different maximum buffers and thus number of points that can be scanned in one scan task varies on available buffer sizes not withstanding protocol limitations. Since Adroit creates scan tasks/jobs automatically, it is important to choose a PLC type that would not result in scan task requesting for more data in one scan than that can be handled by the protocol.

Consult the list below to choose the most appropriate PLC Type.

From revision 25 onwards there is a "CUSTOM" CPU type selection where the PDU lengths are individually configurable in the advanced dialogue (See Advanced button).
The maximum number of items (Digital bits or 16-bit registers) that can be communicated for the different Modicon PLC's is shown below:

	84	884	184	384	484	584	984
000001/%Q0	64	1984	800	800	512	1984	1984
100001/%I0	64	1984	800	800	512	1984	1984
300001/%IW0	4	124	100	100	32	124	124
400001/%MW0	32	124	100	100	124	124	124
600000/%ME0	32	123	100	100	123	123	123

	Custom	Quantum	Premium	M340
000001/%Q0	*	1984	1984	1984
100001/%I0	*	1984	1984	1984
300001/%IW0	*	124	124	124
400001/%MW0	*	124	124	124
600000/%ME0	*	123	123	123

If the PLC slave is not one of those in the list, and is emulating MODBUS, use this table to determine a match for the maximum telegram lengths supported by PLC. Selecting shorter, telegram lengths giver slower performance due to more scan tasks, but ensures compatibility.

* PDU lengths based on vendor-supplied information see "Advanced Dialog".

4. Protocol Definition and Considerations

4.1. Address Padding.

Normal addresses from 1 – 254, ID occupies 1 byte.

Slave/Unit ID
01

For addresses from 255 – 65535 the address is 'padded' with 0XFF and the ID now occupies two bytes.

Padding	Slave Hi ID	Slave Lo ID
FF	xx	xx

Note:

4.1.1. TCP/IP Header Structure (Prefix structure for ALL messages)

Note: There is no CRC field present in the TCP/IP data frame, message integrity is catered for in the TCP/IP protocol and connection.

TCP header:

Transaction Hi ref	Transaction Lo ref	Protocol ID	Protocol ID	Hi Len	Lo Len	Slave/Unit ID
00	00	00	00	00	XX	01

Transaction reference number field is used in since revision 34, please see the Advanced Settings dialog for more detail. The unit ID is shown here as part of the header for illustration purposes only. Some devices may ignore the unit ID or station-# field, or use a default of 1.

4.1.2. Coils 000001- 065535 (%Q0-%Q65535)

Read multiple coils 01 request:

Function	Start Addr Hi	Start Addr Lo	Bit count Hi	Bit count Lo
01	00	01	00	01

Read response:

Function	Byte Count	Data
01	01	01

Exception error response: (see table below for error definition)

Function	Code
81	02

Example: Read 1 coil at reference 0 (000001) resulting in value 1.

Write single coil 05

Write request single:

Function	Coil Addr Hi	Coil Addr Lo	Force Data Hi	Force Data Lo
05	00	00	FF	00

Write response:

Function	Coil Addr Hi	Coil Addr Lo	Force Data Hi	Force Data Lo
05	00	00	FF	00

Exception error response: (see table below for error definition)

Function	Code
85	02

Example: Write 1 coil at reference 0 (000001) resulting in 0 -> 1
(FF00 = true/on) (0000 = False/off)

Write Multiple / Force multiple 0F (i.e. Coils mapped as integer):

Function	Coil Addr Hi	Coil Addr Lo	Bit count Data Hi	Bit count Data Lo	Byte Count	Force Data Hi	Force Data Lo
0f	00	00	00	0f	01	12	34

Write response:

Function	Coil Addr Hi	Coil Addr Lo	Bit count Data Hi	Bit count Data Lo
0f	00	00	00	0f

Exception error response: (see table below for error definition)

Function	Code
8f	02

Example: Write 16 coils at reference 0 (000001) through (000016) resulting in
<- MSB 0000000001111100 -> LSB -- note warning above!

4.1.3. Inputs 100001- 165535 (%I0-%I65535)

Read relays 02 request:

Function	Start Addr Hi	Start Addr Lo	Bit count Hi	Bit count Lo
02	00	00	00	01

Read response:

Function	Byte Count	Data
02	01	01

Exception error response: (see table below for error definition)

Function	Code
82	02

Example: Read 1 discrete input at reference 0 (100001) resulting in value 1

Write relays request:
Not supported in MODBUS protocol, and normally it is not possible to write to an input.

4.1.4. Input Registers 300001- 365535 (%IW0-%IW65535)

Read request:

Function	Start Addr Hi	Start Addr Lo	Word count Hi	Word count Lo
04	00	00	00	01

Read response:

Function	Byte Count	Data Hi	Data Lo
04	02	12	34

Exception error response: (see table below for error definition)

Function	Code
84	02

Example: Read 1 input register at reference 0 (300001) resulting in value 1234 hex

Write request:
Not supported in MODBUS protocol

4.1.5. Holding Registers 400001- 465535 (%MW0-%MW65535)

Read multiple holding registers 03 request:

Function	Start Addr Hi	Start Addr Lo	Word count Hi	Word count Lo
03	00	00	00	01

Read response:

Function	Byte Count	Data Hi	Data Lo
03	02	12	34

Exception error response: (see table below for error definition)

Function	Code
83	02

Example: Read 1 register at reference 0 (400001) resulting in value 1234 hex

Write holding register 10 request:

Function	Start Addr Hi	Start Addr Lo	Word count Hi	Word count Lo	Byte count	Data Hi	Data Lo
10	00	00	00	01	02	12	34

Write response:

Function	Start Addr Hi	Start Addr Lo	Word count Hi	Word count Lo
10	00	13	00	02

Exception error response: (see table below for error definition)

Function	Code
90	02

Example: Write 1 register at reference 0 (400001) to 1234 hex

Write/preset single register 06 request:

Not shown, this frame (command 06) will be used in preference to the Write multiple registers (command 09) only when writing a single 16-bit tag, and if the option to do so is turned on.

4.1.6. Extended Registers 600001- 498303(NB 600000 is valid) (%ME0-%ME98303)

Read request:

Function	Remainder Count	Register Area	File Addr Hi	File Addr Lo	Offset Addr Hi	Offset Addr Lo	Word count Hi	Word count Lo
14	07	06	00	01	00	02	00	01

Read response:

Function	Overall Count	Byte Count	Register Area	Data Hi	Data Lo
14	04	03	06	12	34

Exception error response: (see table below for error definition)

Function	Code
94	02

Example: Read 1 register at reference 1,2 (File 1, offset 2: 600001) resulting in value 1234 hex
(Note: File 1, Offset 0 = 600000)

Write multiple registers 09 request:

Function	Remainder Count	Register Area	File Addr Hi	File Addr Lo	Offset Addr Hi	Offset Addr Lo	Word count Hi	Word Count Lo	Data hi	Data Lo
15	09	06	00	01	00	02	00	01	12	34

Write response:

Function	Remainder Count	Register Area	File Addr Hi	File Addr Lo	Offset Addr Hi	Offset Addr Lo	Word count Hi	Word Count Lo	Data hi	Data Lo
15	09	06	00	01	00	02	00	01	12	34

Exception error response: (see table below for error definition)

Function	Code
95	02

Example: Write 1 register at reference 1,2 (File 1 Offset 2 : 600001) to 1234 hex
Note: File 1 ,Offset 0 = 600000

5. General Notes and Troubleshooting

5.1. Contact Details

Email: support@adroit.co.za and drivers@adroit.co.za

Phone: +27 11 658-8100

Web: www.adroit.co.za

5.2. Adroit Technologies Knowledge Base

For additional information, please refer to our Knowledgebase website:

Latest Drivers topics - Features, discussions, tips, tricks, questions, problems and feedback (adroittechnologiesautomation.com)

5.3. PLC Exception Codes

These codes are returned by the PLC. Some action either in scan configuration, driver setup parameters or hardware configuration may be needed to rectify the problem.

Hex code	Meaning	Description
0x0	No error	
0x1	Illegal Function	The function code received in the query is not an allowable action for the slave.
0x2	Illegal Data Address	The data address received is not an allowable address for the slave.
0x3	Illegal Data value	A value contained in the query data field is not an allowable value for the slave.
0x4	Slave Device failure	Unrecoverable error occurred in the slave while processing the request.
0x5	Acknowledge	This response is returned if the slave is processing the request but will take a long time; this is to prevent the master from timing out.
0x6	Slave device busy	Re-transmit the command later when the slave is not busy.
0x7	Negative acknowledge	Slave cannot perform a programming function requested.
0x8	Memory Parity error	The slave may require service. This error may occur when a request is made to read extended memory and a parity error is detected in the memory.
0xA	Gateway Path un-available	The remote device on a Modbus plus + network did not respond to the gateway in time, this may occurs if a device routes through the gateway onto a different network and is no longer connected.
0xB	Gateway Target failed to respond.	The remote device did not respond to the gateway in time, this occurs if a device routes through a gateway onto a

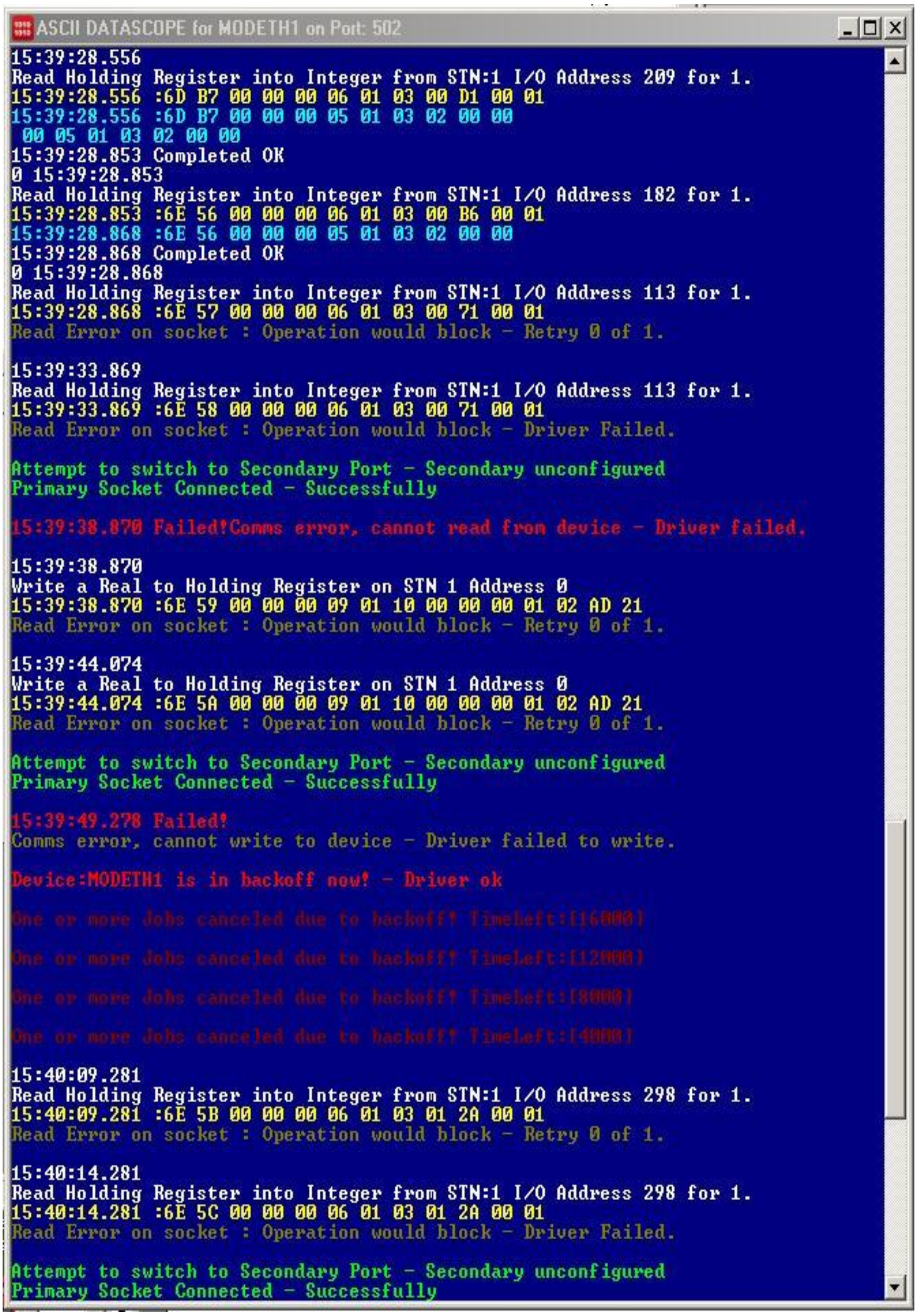
		different network such as RS485, and is no longer connected.

5.4. Diagnostic Traces

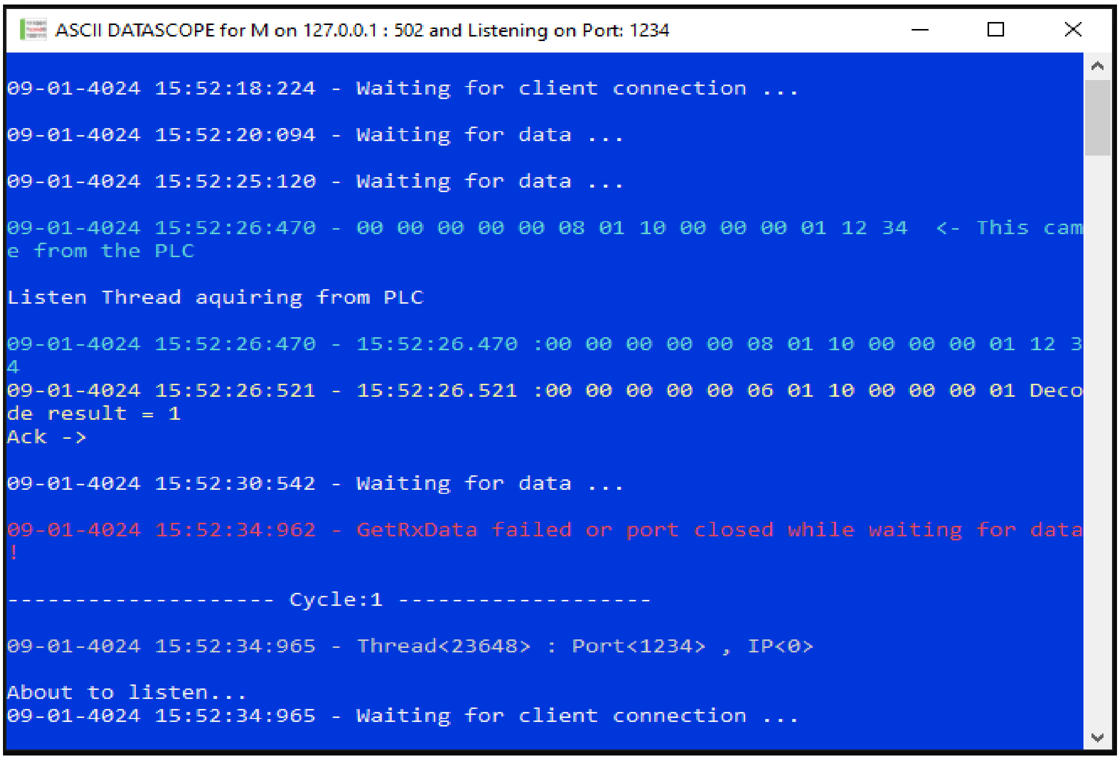
Action	State	Meaning
SEQ word error, nnnn, expected nnnn	Response was out of sequence	The reference filed in the header of the TCP frame was not correctly copied by the slave. This occurs if a slave responds too late to our request, and hence goes out of sync. If you see this message and get no comms at all, you may want to turn off the frame sequence numbering option in the advanced settings.
Cannot create file handle for driver monitoring	Driver not operational	1. Check that you have at least 500kb free disk space. 2. Check that you have file create rights on the working-directory. 3. A folder with the same name as the Device Agent exists in the working directory.
Invalid BCD value from PLC (max value = 9999)	Driver not able to parse return message from PLC	A request for a range of BCD values included some non-valid entries; try ensuring all BCD data is packed sequentially into registers, and that they are all valid.
NNN bytes of noise purged	Noise Purged OK	Data was received unexpectedly over the connection; this may occur after a timeout if the slave responds too late.
PLC responded with exception - zzzzzz	Driver will retry	See the table above for a description of the exception codes. These codes are codes returned by the PLC, not driver errors.
Successful write to device/ Successful read from device	Driver healthy.	The driver has recovered from an error, either of these 2 messages will appear only once following any prior error.
Primary Socket initialised; Secondary Socket not initialised	Device zzzz healthy	This 'informational' event will appear in the logs at start-up and indicates that the Windows TCP/IP protocol stack is in working order.

5.5. Datascope examples for reference

Example of the driver datascope.

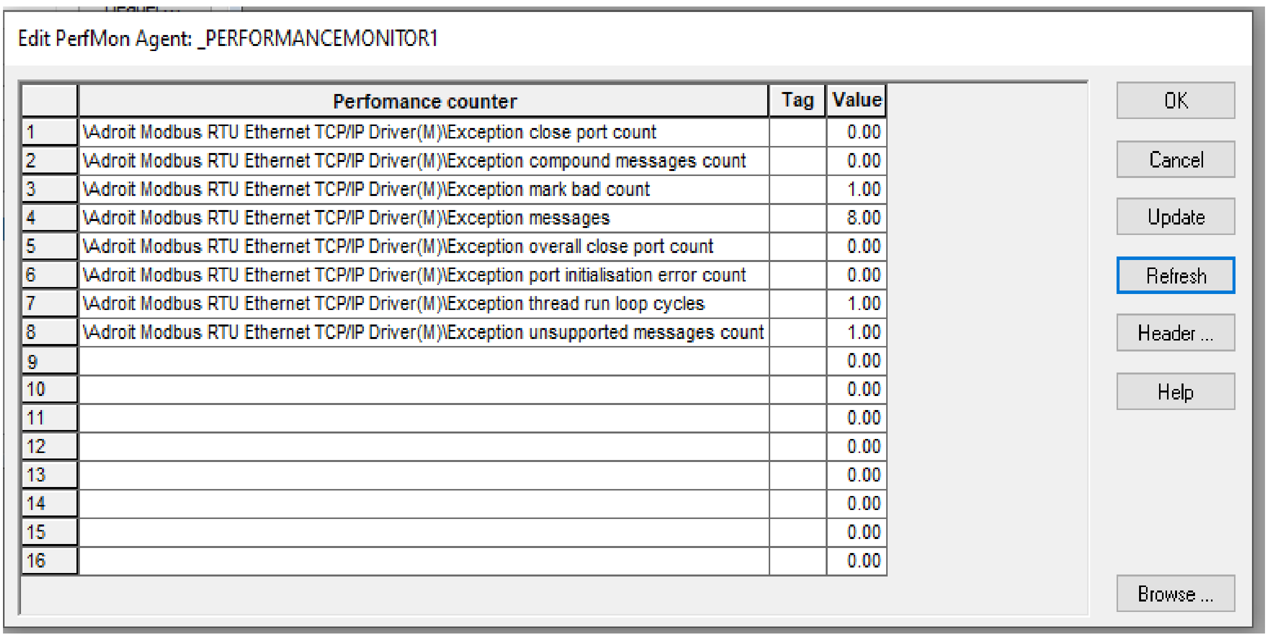


Example of the driver running with Unsolicited listen mode enabled. Note the port listening information on the title bar.



5.6. Performance counters

Along with the normal driver counters (not shown here) the are additional Unsolicited/Exception counters available to push to tags via the performance agent for alarming and monitoring purposes.



	Perfomance counter	Tag	Value
1	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception close port count		0.00
2	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception compound messages count		0.00
3	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception mark bad count		1.00
4	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception messages		8.00
5	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception overall close port count		0.00
6	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception port initialisation error count		0.00
7	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception thread run loop cycles		1.00
8	\Adroit Modbus RTU Ethernet TCP/IP Driver(M)\Exception unsupported messages count		1.00
9			0.00
10			0.00
11			0.00
12			0.00
13			0.00
14			0.00
15			0.00
16			0.00

6.Revisions

6.1 Document Revision

Rev.	Date	Author	Comment	Approval
1.0	14-Dec-2022	J. Duncan	New Format	D. Jordaan
1.1	16-Jan-2023	K. Robinson	Edited content, aligned images and grammar changes.	D. Jordaan
2.0	02-Feb-2023	D. Jordaan	Updated documentation as per Rev 99	-
3.0	09-Jan-2024	D. Jordaan	Updated documentation as per Rev 103	-
4.0				
5.0				
6.0				

6.2 DLL Revision

Rev.	Date	Changes
97	10-Oct-2021	Added ResetConnection and SwitchConnection functionality
98	10-Aug-2022	String job alignment may have been misaligned under rare conditions.
99	14-Sep-2022	NetLatencies not operating as expected, if ClosePorAfterEveryTransaction was also checked, writes would fail to reconnect, symptoms included no visible writes on the datascope and affected jobs would intermittently be toggling between bad and good status when writes were due.
100,101	12-Dec-2023	Critical fix, 30000x Sy or Cy and RAWBYTES jobs incorrectly retrieved data from 40000x file.
102,103	03-Jan-2024	Added Unsolicited listen mode, More datascope info , Dynamic config mode added to device stop call – ready for the restart. Dynamic config will also be attempted in the listen algorithm. Added unsolicited specif perfmance counters.